

SBA301

HƯỚNG DẪN DỰ ÁN TỔNG HỢP SLOT 07

Campus Event Navigator

React Router & Navigation — Routes, Links, Parameters & SPA Navigation

Mục tiêu trung tâm

Xây dựng một React SPA mức độ trung bình trên IntelliJ IDEA, có nhiều URL/view, navigation bằng React Router, dynamic detail route, useParams, useNavigate, NotFound, browser history và kiểm chứng route bằng evidence.

Thông tin	Giá trị
Course	SBA301 – Integrate Single Page Application with Spring Boot
Slot	07 – củng cố Prompt 06 / Chapter 06
Project	Campus Event Navigator
Độ khó	Trung bình – frontend SPA, static data, no backend
IDE	IntelliJ IDEA
Core stack	ReactJS, Vite, React-Bootstrap, React Router DOM, JavaScript, Git
Core routes	/, /events, /events/:id, /about, *
Kiến thức tái sử dụng	Component, Props, useState, map/key, React-Bootstrap
Chưa dùng	Axios, REST API, Spring Boot, database, auth guards

Nguyên tắc thực hành

DESIGN URL FIRST → BUILD SMALL → RUN → VERIFY URL + VIEW → DEBUG WITH EVIDENCE → COMMIT → EXPLAIN

1. TÓM TẮT DỰ ÁN & MỤC TIÊU

Campus Event Navigator là một ứng dụng SPA cho phép sinh viên xem danh sách sự kiện trong trường, lọc nhanh dữ liệu, mở trang chi tiết bằng URL động và điều hướng giữa nhiều view mà không biến mỗi view thành một HTML file riêng.

Mental model cốt lõi

ONE REACT APP + MULTIPLE URLS + MULTIPLE VIEWS = SPA ROUTING

1.1 Mục tiêu học tập

- Hiểu routing là mapping từ URL đến rendered view.
- Cấu hình BrowserRouter, Routes và Route.
- Dùng Link cho điều hướng nội bộ; dùng useNavigate khi logic/event chủ động điều hướng.
- Tạo dynamic route `/events/:id` và đọc `id` bằng useParams.
- Tìm dữ liệu local bằng route parameter và xử lý event không tồn tại.
- Phân biệt client route với REST endpoint tương lai.
- Kiểm tra Browser Back/Forward, direct URL, bookmarkable URL và NotFound.
- Ôn Props, useState, map(), key, component architecture và React-Bootstrap.

1.2 Kết quả đầu ra phải chứng minh

Evidence	Yêu cầu
Application	Chạy bằng Vite, không có blocking error
Route table	Ít nhất 5 route patterns
Collection	/events hiển thị danh sách bằng reusable EventCard
Dynamic detail	/events/:id hiển thị đúng record
Invalid resource	/events/999 không crash
Invalid route	URL bất kỳ đi vào NotFound
Navigation	Link + một useNavigate use case
History	Back/Forward hoạt động hợp lý
Git	Có verified commit
Explanation	Sinh viên giải thích URL → Route → View → parameter → lookup

2. CHỨC NĂNG DỰ ÁN

#	Chức năng	Route/Mechanism	Mức
1	Home giới thiệu ứng dụng	/	Core
2	Danh sách sự kiện	/events	Core
3	Tìm kiếm theo tiêu đề/địa điểm	useState + derived filter	Reinforcement
4	Lọc category	useState	Reinforcement
5	Card tái sử dụng	Props + map/key	Reinforcement
6	Chi tiết sự kiện	/events/:id	Core
7	Đọc id từ URL	useParams	Core
8	Nút Explore Events	useNavigate	Intro practice
9	Back to Events / Go Back	Link + useNavigate	Core/Intro
10	Event ID không tồn tại	Resource Not Found	Core
11	Route không tồn tại	path=""	Core

#	Chức năng	Route/Mechanism	Mức
12	Browser history	Back/Forward	Core
13	Direct URL / bookmark	/events/3	Core
14	Responsive Navbar/Grid	React-Bootstrap	Reinforcement

3. CÔNG NGHỆ SỬ DỤNG

Công nghệ	Vai trò trong dự án	Kiến thức Slot
ReactJS	Component-based UI	Components, Props, State
Vite	Scaffold + dev server + build	Tooling
react-router-dom	Client-side routing	Slot 07 core
React-Bootstrap	Navbar, Grid, Card, Button, Badge, Alert	UI reuse
JavaScript	Array map/filter/find, template literals	Data/render logic
IntelliJ IDEA	IDE, Terminal, npm Run Configuration	Development environment
Browser DevTools	Console + URL verification	Debug evidence
Git	Working checkpoint	Engineering habit

Không có backend

events.js chỉ là static frontend data. React Router chọn view; nó KHÔNG tự gọi Spring Boot và KHÔNG đọc database.

4. KIẾN TRÚC ỨNG DỤNG

4.1 Component & Routing Tree



4.2 Route Table

URL Pattern	Type	View	Dữ liệu/Parameter
/	Static	Home	—
/events	Static	EventsPage	events array
/events/:id	Dynamic	EventDetail	id
/about	Static	About	—
*	Wildcard	NotFound	unknown route

4.3 Data Flow

```

events.js
  ↓ import
EventsPage
  ↓ filter/map
EventCard(event prop)
  ↓ Link /events/{id}
Browser URL
  ↓ Route :id
EventDetail
  ↓ useParams()
  ↓ find()
Selected event
  ↓ render detail

```

Phân biệt library

useParams() = React Router. find()/filter()/map() = JavaScript Array methods.

5. CẤU TRÚC THƯ MỤC MỤC TIÊU

```

sba301-event-navigator/
├── package.json
├── vite.config.js
├── public/
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── index.css
    ├── data/
    │   └── events.js
    ├── components/
    │   ├── AppNavbar.jsx
    │   ├── EventCard.jsx
    │   └── AppFooter.jsx
    └── pages/
        ├── Home.jsx
        ├── EventsPage.jsx
        ├── EventDetail.jsx
        ├── About.jsx
        └── NotFound.jsx

```

`pages/` là convention tổ chức source để nhận biết route-level views; đây không phải yêu cầu kỹ thuật bắt buộc của React Router.

6. TÓM TẮT 30 BƯỚC THỰC HIỆN

Bước	Nội dung	Gate
0	Chuẩn bị & kiểm tra Node/npm/Git	PASS trước khi sang bước sau
1	Mở/tạo project trong IntelliJ	PASS trước khi sang bước sau
2	Baseline Vite chạy được	PASS trước khi sang bước sau
3	Cài React-Bootstrap + React Router	PASS trước khi sang bước sau
4	Cấu hình Bootstrap CSS	PASS trước khi sang bước sau
5	Dọn starter code	PASS trước khi sang bước sau
6	Tạo folder structure	PASS trước khi sang bước sau
7	Tạo static events data	PASS trước khi sang bước sau
8	Tạo AppNavbar	PASS trước khi sang bước sau
9	Tạo AppFooter	PASS trước khi sang bước sau
10	Tạo Home page	PASS trước khi sang bước sau
11	Tạo EventCard	PASS trước khi sang bước sau
12	Tạo EventsPage + search/filter	PASS trước khi sang bước sau
13	Tạo About page	PASS trước khi sang bước sau
14	Tạo EventDetail + useParams	PASS trước khi sang bước sau
15	Tạo NotFound	PASS trước khi sang bước sau
16	Cấu hình App routes	PASS trước khi sang bước sau
17	Bọc App bằng BrowserRouter	PASS trước khi sang bước sau
18	Verify static routes bằng URL	PASS trước khi sang bước sau
19	Verify Navbar Link navigation	PASS trước khi sang bước sau
20	Verify dynamic detail URLs	PASS trước khi sang bước sau
21	Verify resource-not-found	PASS trước khi sang bước sau
22	Verify wildcard NotFound	PASS trước khi sang bước sau
23	Verify useNavigate	PASS trước khi sang bước sau
24	Verify browser history/direct URL	PASS trước khi sang bước sau
25	Hoàn thiện CSS/responsive	PASS trước khi sang bước sau
26	Tạo IntelliJ npm Run Configuration	PASS trước khi sang bước sau
27	Chạy manual test matrix	PASS trước khi sang bước sau
28	Debug theo decision tree	PASS trước khi sang bước sau
29	Git + production build/preview	PASS trước khi sang bước sau

7. HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

Mỗi bước đều có ACTION – WHY – EXPECTED – VERIFY – IF FAILED.

Bước 0. Kiểm tra môi trường

Mục	Hướng dẫn
ACTION	Mở IntelliJ Terminal. Chạy `node -v`, `npm -v`, `git --version`.
WHY	Bảo đảm toolchain tồn tại trước khi scaffold/cài package.
EXPECTED RESULT	Ba lệnh trả version, không báo command not found.
HOW TO VERIFY	Chụp/ghi lại version; không cần giống tuyệt đối máy bạn khác.
IF FAILED	Sửa PATH/Node interpreter theo tài liệu setup; không bắt đầu code khi npm chưa chạy.

```
node -v
npm -v
git --version
```

Bước 1. Mở hoặc tạo project trong IntelliJ

Mục	Hướng dẫn
ACTION	Khuyến nghị: mở project EventHub/React working project của Slot 05 để refactor sang routing. Nếu tạo mới, dùng File → Open sau khi scaffold folder.
WHY	Routing nên được thêm vào một React baseline đã hoạt động.
EXPECTED RESULT	Project tree có `src`, `package.json`; Terminal đang ở đúng root.
HOW TO VERIFY	Chạy `pwd` / `Get-Location`; kiểm tra `package.json`.
IF FAILED	Nếu Terminal đang sai folder, `cd` về project root.

Nếu tạo project mới

Dùng Vite version đã được giảng viên/course environment phê duyệt. Nếu môi trường hiện tại hỗ trợ `npm create vite@latest`, có thể scaffold React template; nếu Node bị khóa và @latest báo engine incompatibility, KHÔNG nâng Node ngẫu nhiên—dùng Vite version đã khóa trong hướng dẫn môi trường.

```
# Ví dụ scaffold khi môi trường cho phép
npm create vite@latest sba301-event-navigator -- --template react
cd sba301-event-navigator
npm install
```

Bước 2. Baseline Vite

Mục	Hướng dẫn
ACTION	Chạy `npm run dev` trước khi thêm Router.
WHY	Tách lỗi baseline khỏi lỗi do routing.
EXPECTED RESULT	Terminal hiện local URL và browser mở app.
HOW TO VERIFY	Mở Console: không có blocking error.
IF FAILED	Đọc lỗi đầu tiên; kiểm tra `npm install`, đúng directory, package.json.

```
npm run dev
```

Bước 3. Cài dependencies

Mục	Hướng dẫn
ACTION	Trong project root chạy `npm install react-bootstrap bootstrap react-router-dom`.
WHY	Dự án cần Bootstrap components/CSS và React Router APIs.
EXPECTED RESULT	Install hoàn tất không có blocking error.
HOW TO VERIFY	Mở package.json xác nhận 3 dependencies.
IF FAILED	Nếu module not found: kiểm tra directory và npm install output.

```
npm install react-bootstrap bootstrap react-router-dom
```

Bước 4. Import Bootstrap CSS

Mục	Hướng dẫn
ACTION	Mở `src/main.jsx`; import Bootstrap CSS trước custom CSS.
WHY	React-Bootstrap package không tự import stylesheet.
EXPECTED RESULT	UI Bootstrap hiển thị đúng style.
HOW TO VERIFY	App vẫn compile sau import.
IF FAILED	Nếu module CSS not found, xác nhận package `bootstrap` đã được cài.

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Bước 5. Dọn starter code

Mục	Hướng dẫn
ACTION	Xóa demo Vite không còn cần (logo/counter markup), giữ cấu trúc React tối thiểu.
WHY	Giảm nhiễu khi debug Router.
EXPECTED RESULT	App có thể tạm hiển thị một heading đơn giản.
HOW TO VERIFY	Save → browser HMR cập nhật.
IF FAILED	Nếu blank screen, xem Console trước khi tiếp tục.

Bước 6. Tạo folder structure

Mục	Hướng dẫn
ACTION	Trong `src`, tạo `data`, `components`, `pages`; tạo các file đúng cây mục tiêu.
WHY	Separation of responsibilities giúp route/view rõ ràng.
EXPECTED RESULT	Project tree khớp cấu trúc ở mục 5.
HOW TO VERIFY	IntelliJ Project tool window hiển thị đúng tên/case.
IF FAILED	Sửa tên file trước khi import để tránh path mismatch.

Bước 7. Tạo static event data

Mục	Hướng dẫn
ACTION	Tạo `src/data/events.js` với 8 records. ID dùng STRING để khớp trực tiếp với URL parameter.
WHY	Chuẩn hóa nguồn dữ liệu cho list và detail lookup.

Mục	Hướng dẫn
EXPECTED RESULT	File export mảng `events`; không syntax error.
HOW TO VERIFY	IntelliJ không báo đỏ; save file.
IF FAILED	Kiểm tra dấu phẩy, quote, export.

```
// src/data/events.js
export const events = [
  {
    id: "1",
    title: "AI & Software Engineering Day",
    category: "Technology",
    date: "2026-10-03",
    location: "Innovation Hall A",
    organizer: "Computing Club",
    featured: true,
    seats: 180,
    description: "Talks and demos on AI-assisted software engineering workflows."
  },
  {
    id: "2",
    title: "React Front-end Workshop",
    category: "Workshop",
    date: "2026-10-08",
    location: "Lab 204",
    organizer: "Web Engineering Team",
    featured: true,
    seats: 45,
    description: "Hands-on component, state, routing and debugging practice."
  },
  {
    id: "3",
    title: "Startup Pitch Evening",
    category: "Business",
    date: "2026-10-12",
    location: "Auditorium",
    organizer: "Entrepreneurship Center",
    featured: false,
    seats: 220,
    description: "Student teams present product ideas and receive feedback."
  },
  {
    id: "4",
    title: "Cybersecurity Capture the Flag",
    category: "Competition",
    date: "2026-10-17",
    location: "Lab 301",
    organizer: "Security Club",
    featured: true,
    seats: 80,
    description: "Team-based security challenges for beginner and intermediate learners."
  },
  {
    id: "5",
    title: "English Presentation Clinic",
    category: "Skills",
    date: "2026-10-21",
    location: "Room B105",
    organizer: "Academic Skills Unit",
    featured: false,
    seats: 60,
    description: "Practice structure, delivery and technical presentation skills."
  },
  {
    id: "6",
    title: "Mobile App Demo Day",
    category: "Technology",
    date: "2026-10-25",
    location: "Innovation Hall B",
```

```

    organizer: "Mobile Lab",
    featured: false,
    seats: 150,
    description: "Showcase of student mobile applications and engineering decisions."
  },
  {
    id: "7",
    title: "Database Design Challenge",
    category: "Competition",
    date: "2026-11-01",
    location: "Lab 205",
    organizer: "Data Club",
    featured: false,
    seats: 70,
    description: "Model data requirements and defend schema design decisions."
  },
  {
    id: "8",
    title: "Research Poster Forum",
    category: "Research",
    date: "2026-11-06",
    location: "Library Commons",
    organizer: "Research Office",
    featured: true,
    seats: 200,
    description: "Poster presentations from student and faculty research projects."
  }
];

```

Bước 8. Tạo AppNavbar

Mục	Hướng dẫn
ACTION	Tạo reusable navigation component và tích hợp React-Bootstrap `Nav.Link as={Link}`.
WHY	Navbar là UI; Link mới là Router-aware navigation.
EXPECTED RESULT	Brand/Home/Events/About hiển thị, click không crash.
HOW TO VERIFY	Click sau khi routes được tạo; lúc này chỉ cần compile.
IF FAILED	Kiểm tra import `Link`, tên component, closing tags.

```

// src/components/AppNavbar.jsx
import { Container, Nav, Navbar } from "react-bootstrap";
import { Link } from "react-router-dom";

function AppNavbar() {
  return (
    <Navbar bg="dark" data-bs-theme="dark" expand="lg">
      <Container>
        <Navbar.Brand as={Link} to="/">Campus Event Navigator</Navbar.Brand>
        <Navbar.Toggle aria-controls="main-nav" />
        <Navbar.Collapse id="main-nav">
          <Nav className="ms-auto">
            <Nav.Link as={Link} to="/">Home</Nav.Link>
            <Nav.Link as={Link} to="/events">Events</Nav.Link>
            <Nav.Link as={Link} to="/about">About</Nav.Link>
          </Nav>
        </Navbar.Collapse>
      </Container>
    </Navbar>
  );
}

export default AppNavbar;

```

Bước 9. Tạo AppFooter

Mục	Hướng dẫn
ACTION	Tạo footer độc lập để App chỉ chịu trách nhiệm composition/layout.
WHY	Củng cố component architecture từ Slot 03.
EXPECTED RESULT	Footer render ở mọi route.
HOW TO VERIFY	Sau khi App được ghép, chuyển route và xác nhận footer vẫn còn.
IF FAILED	Kiểm tra import/export.

```
// src/components/AppFooter.jsx
function AppFooter() {
  return (
    <footer className="app-footer">
      SBA301 • Campus Event Navigator • React Router Practice
    </footer>
  );
}

export default AppFooter;
```

Bước 10. Tạo Home page + useNavigate

Mục	Hướng dẫn
ACTION	Tạo Home route-level view. Nút Explore Events dùng `useNavigate()` để minh họa programmatic navigation.
WHY	Cho thấy Link và useNavigate có use case khác nhau.
EXPECTED RESULT	Button click đưa URL sang `/events`.
HOW TO VERIFY	Quan sát address bar và rendered EventsPage.
IF FAILED	Nếu hook error: chắc chắn Home nằm trong BrowserRouter và useNavigate được import.

```
// src/pages/Home.jsx
import { Button, Container } from "react-bootstrap";
import { useNavigate } from "react-router-dom";

function Home() {
  const navigate = useNavigate();

  return (
    <Container className="py-5 text-center">
      <h1>Campus Event Navigator</h1>
      <p className="lead">Discover technology, research, business and skills events.</p>
      <Button onClick={() => navigate("/events")}>Explore Events</Button>
    </Container>
  );
}

export default Home;
```

Bước 11. Tạo EventCard

Mục	Hướng dẫn
ACTION	Tạo một reusable Card nhận `event` prop và Link sang dynamic detail URL.
WHY	Một component + nhiều data objects, đồng thời nối collection với routing.
EXPECTED RESULT	Mỗi card có title/category/date/location/View Details.

Mục	Hướng dẫn
HOW TO VERIFY	Render thử 1 card trước khi map toàn bộ.
IF FAILED	Nếu Link sai, kiểm tra template literal và `event.id`.

```
// src/components/EventCard.jsx
import { Badge, Button, Card } from "react-bootstrap";
import { Link } from "react-router-dom";

function EventCard({ event }) {
  return (
    <Card className="h-100 shadow-sm">
      <Card.Body className="d-flex flex-column">
        <div className="mb-2">
          <Badge bg={event.featured ? "warning" : "secondary"}>
            {event.featured ? "Featured" : event.category}
          </Badge>
        </div>
        <Card.Title>{event.title}</Card.Title>
        <Card.Text className="mb-1"><strong>Date:</strong> {event.date}</Card.Text>
        <Card.Text className="mb-3"><strong>Location:</strong> {event.location}</Card.Text>
        <Button
          as={Link}
          to={` /events/${event.id}`}
          className="mt-auto"
        >
          View Details
        </Button>
      </Card.Body>
    </Card>
  );
}

export default EventCard;
```

Bước 12. Tạo EventsPage với search/filter

Mục	Hướng dẫn
ACTION	Dùng local useState cho search/category; derived list bằng filter; render bằng map + stable key.
WHY	Ôn Slot 04–05 nhưng routing vẫn là mục tiêu chính.
EXPECTED RESULT	/events hiển thị 8 cards ban đầu; filter làm giảm count.
HOW TO VERIFY	Gõ keyword và đổi category; Console không lỗi.
IF FAILED	Trace data → filter → map → EventCard; không đổ lỗi Router cho filter bug.

```
// src/pages/EventsPage.jsx (phần 1)
import { useState } from "react";
import { Col, Container, Form, Row } from "react-bootstrap";
import EventCard from "../components/EventCard";
import { events } from "../data/events";

function EventsPage() {
  const [search, setSearch] = useState("");
  const [category, setCategory] = useState("All");

  const categories = ["All", ...new Set(events.map(e => e.category))];
  const filteredEvents = events.filter((event) => {
    const keyword = search.trim().toLowerCase();
    const matchesText =
      event.title.toLowerCase().includes(keyword) ||
      event.location.toLowerCase().includes(keyword);
    const matchesCategory = category === "All" || event.category === category;
    return matchesText && matchesCategory;
  });
}
```

```
// src/pages/EventsPage.jsx (phần 2)
return (
  <Container className="py-4">
    <h2>Campus Events</h2>
    <p className="text-muted">{filteredEvents.length} event(s) found.</p>

    <Row className="g-3 mb-4">
      <Col md={8}>
        <Form.Control
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          placeholder="Search title or location..."
        />
      </Col>
      <Col md={4}>
        <Form.Select value={category} onChange={(e) => setCategory(e.target.value)}>
          {categories.map((item) => (
            <option key={item} value={item}>{item}</option>
          ))}
        </Form.Select>
      </Col>
    </Row>

    <Row className="g-4">
      {filteredEvents.map((event) => (
        <Col key={event.id} sm={12} md={6} lg={4}>
          <EventCard event={event} />
        </Col>
      ))}
    </Row>

    {filteredEvents.length === 0 && (
      <p className="text-center text-muted mt-4">No events found.</p>
    )}
  </Container>
);
}

export default EventsPage;
```

Bước 13. Tạo About page

Mục	Hướng dẫn
ACTION	Tạo route-level static view đơn giản.
WHY	Có ít nhất ba static views để kiểm tra navigation.

Mục	Hướng dẫn
EXPECTED RESULT	/about render heading và mô tả.
HOW TO VERIFY	Gõ direct URL `/about`.
IF FAILED	Nếu không render, kiểm tra route config thay vì CSS.

```
// src/pages/About.jsx
import { Container } from "react-bootstrap";

function About() {
  return (
    <Container className="py-5">
      <h2>About</h2>
      <p>Campus Event Navigator is an SBA301 routing practice application.</p>
      <p>Current data is local JavaScript data; no backend request is performed.</p>
    </Container>
  );
}

export default About;
```

Bước 14. Tạo EventDetail với useParams

Mục	Hướng dẫn
ACTION	Đọc `id` từ URL, tìm record bằng `find()`, render detail và xử lý resource not found.
WHY	Đây là core dynamic routing skill.
EXPECTED RESULT	/events/1 và /events/2 hiển thị hai dữ liệu khác nhau; /events/999 không crash.
HOW TO VERIFY	Tạm `console.log(id)` nếu cần; xác nhận id là string.
IF FAILED	Kiểm tra route parameter name, ID type và find comparison.

```
// src/pages/EventDetail.jsx (phần 1)
import { Alert, Badge, Button, Card, Container } from "react-bootstrap";
import { Link, useNavigate, useParams } from "react-router-dom";
import { events } from "../data/events";

function EventDetail() {
  const { id } = useParams();
  const navigate = useNavigate();
  const event = events.find((item) => item.id === id);

  if (!event) {
    return (
      <Container className="py-5">
        <Alert variant="warning">
          <Alert.Heading>Event not found</Alert.Heading>
          <p>No local event has id: <strong>{id}</strong>.</p>
          <Button as={Link} to="/events">Back to Events</Button>
        </Alert>
      </Container>
    );
  }
}
```

```
// src/pages/EventDetail.jsx (phần 2)
return (
  <Container className="py-5">
    <Card className="shadow-sm">
      <Card.Body>
        <div className="mb-3">
          <Badge bg={event.featured ? "warning" : "secondary"}>
            {event.category}
          </Badge>
        </div>
        <Card.Title as="h2">{event.title}</Card.Title>
        <Card.Text>{event.description}</Card.Text>
        <hr />
        <p><strong>Date:</strong> {event.date}</p>
        <p><strong>Location:</strong> {event.location}</p>
        <p><strong>Organizer:</strong> {event.organizer}</p>
        <p><strong>Seats:</strong> {event.seats}</p>
        <div className="d-flex gap-2">
          <Button as={Link} to="/events" variant="primary">Back to Events</Button>
          <Button variant="outline-secondary" onClick={() => navigate(-1)}>Go Back</Button>
        </div>
      </Card.Body>
    </Card>
  </Container>
);
}

export default EventDetail;
```

Bước 15. Tạo NotFound page

Mục	Hướng dẫn
ACTION	Tạo client 404 view cho URL không khớp route pattern.
WHY	Phân biệt route-not-found với resource-not-found.
EXPECTED RESULT	/abc123 hiển thị Not Found; /events/999 hiển thị Event not found.
HOW TO VERIFY	Test cả hai URL.
IF FAILED	Nếu wildcard nuốt route đúng, kiểm tra cấu hình Routes và path typo.

```
// src/pages/NotFound.jsx
import { Button, Container } from "react-bootstrap";
import { Link } from "react-router-dom";

function NotFound() {
  return (
    <Container className="py-5 text-center">
      <h2>404 - Page Not Found</h2>
      <p>The client route you requested does not exist.</p>
      <Button as={Link} to="/">Go Home</Button>
    </Container>
  );
}

export default NotFound;
```

Bước 16. Cấu hình App routes

Mục	Hướng dẫn
ACTION	Ghép layout với AppNavbar, Routes/Route và AppFooter.
WHY	App trở thành route composition root.
EXPECTED RESULT	Các path `/`, `/events`, `/events/:id`, `/about`, `*` được khai báo.

Mục	Hướng dẫn
HOW TO VERIFY	Gõ từng URL thủ công sau khi BrowserRouter được thêm.
IF FAILED	Nếu Routes/Route not defined, kiểm tra import từ react-router-dom.

```
// src/App.jsx
import { Route, Routes } from "react-router-dom";
import AppNavbar from "../components/AppNavbar";
import AppFooter from "../components/AppFooter";
import Home from "../pages/Home";
import EventsPage from "../pages/EventsPage";
import EventDetail from "../pages/EventDetail";
import About from "../pages/About";
import NotFound from "../pages/NotFound";

function App() {
  return (
    <div className="app-shell">
      <AppNavbar />
      <main className="app-main">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/events" element={<EventsPage />} />
          <Route path="/events/:id" element={<EventDetail />} />
          <Route path="/about" element={<About />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>
      <AppFooter />
    </div>
  );
}

export default App;
```

Bước 17. Bọc App bằng BrowserRouter

Mục	Hướng dẫn
ACTION	Trong main.jsx, đặt `<App />` bên trong `<BrowserRouter>`.
WHY	Các component dùng Link/useParams/useNavigate cần Router context.
EXPECTED RESULT	Không còn lỗi "may be used only in the context of a Router".
HOW TO VERIFY	Reload app; thử Home button và Navbar.
IF FAILED	Nếu lỗi context, đảm bảo BrowserRouter nằm ngoài App chứ không nằm trong một route child.

```
// src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "bootstrap/dist/css/bootstrap.min.css";
import "../index.css";
import App from "../App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

Bước 18. Verify static routes bằng URL

Mục	Hướng dẫn
ACTION	Gõ lần lượt `/`, `/events`, `/about` trực tiếp trong address bar.
WHY	Tách route matching khỏi Navbar navigation.
EXPECTED RESULT	Mỗi URL render đúng view.
HOW TO VERIFY	Điền Route Verification Matrix.
IF FAILED	Nếu URL đổi nhưng blank, Console + Route path + element import.

URL	Expected	Actual	Result
/	Home		
/events	EventsPage		
/about	About		

Bước 19. Verify Navbar Link

Mục	Hướng dẫn
ACTION	Click Home → Events → About.
WHY	Kiểm tra client navigation từ UI.
EXPECTED RESULT	URL và view cùng thay đổi; layout Navbar/Footer vẫn tồn tại.
HOW TO VERIFY	Quan sát address bar và browser reload indicator.
IF FAILED	Nếu Link chỉ đổi hash/href cũ, kiểm tra Nav.Link `as={Link}` và `to`.

Bước 20. Verify dynamic detail URLs

Mục	Hướng dẫn
ACTION	Từ /events click View Details trên event 1, 2, 3.
WHY	Kiểm tra template literal + dynamic route + route reuse.
EXPECTED RESULT	URL lần lượt `/events/1`, `/events/2`, `/events/3`; cùng một EventDetail render dữ liệu khác nhau.
HOW TO VERIFY	Ghi Card clicked, URL id, rendered title.
IF FAILED	Nếu tất cả card mở cùng event: kiểm tra `event.id` và data prop.

Bước 21. Verify resource-not-found

Mục	Hướng dẫn
ACTION	Gõ `/events/999`.
WHY	Route pattern có match nhưng record không tồn tại.
EXPECTED RESULT	EventDetail hiển thị “Event not found”, không crash.
HOW TO VERIFY	Console không có exception do đọc property của undefined.
IF FAILED	Đảm bảo `if (!event)` xảy ra trước khi render `event.title`.

Bước 22. Verify wildcard NotFound

Mục	Hướng dẫn
-----	-----------

Mục	Hướng dẫn
ACTION	Gõ <code>/something-random`</code> .
WHY	Kiểm tra unknown route handling.
EXPECTED RESULT	NotFound page hiển thị.
HOW TO VERIFY	So sánh với <code>/events/999`</code> và giải thích khác biệt.
IF FAILED	Kiểm tra <code>path="**"</code> tồn tại.

Bước 23. Verify useNavigate

Mục	Hướng dẫn
ACTION	Ở Home click Explore Events; ở EventDetail click Go Back.
WHY	Thực hành programmatic navigation ở mức phù hợp.
EXPECTED RESULT	Explore đưa đến /events; Go Back đi về history entry trước.
HOW TO VERIFY	Thử vào detail từ list, sau đó Go Back.
IF FAILED	Nếu <code>navigate(-1)`</code> về trang ngoài app khi mở direct URL, đây là behavior của history; nút Back to Events bằng Link vẫn là đường dẫn ổn định.

Bước 24. Verify browser history & direct URL

Mục	Hướng dẫn
ACTION	Đi Home → Events → Detail, dùng Back/Forward. Sau đó copy <code>/events/3`</code> sang tab mới trong dev server.
WHY	URL phải là application state có thể điều hướng/bookmark ở mức dev.
EXPECTED RESULT	Back/Forward đổi view đúng; direct dev URL hiển thị detail.
HOW TO VERIFY	Ghi 4 bước history vào worksheet.
IF FAILED	Nếu production refresh 404, đó là hosting/server fallback concern; không kết luận Router code sai.

Bước 25. Hoàn thiện CSS/responsive

Mục	Hướng dẫn
ACTION	Thay <code>src/index.css`</code> bằng stylesheet đơn giản để app shell, home, cards, footer rõ ràng.
WHY	Đảm bảo functional correctness trước cosmetic refinement.
EXPECTED RESULT	Desktop/mobile readable; footer ổn định.
HOW TO VERIFY	Resize browser; inspect overflow.
IF FAILED	Nếu styling mất: kiểm tra Bootstrap CSS import và custom CSS path.

```

/* src/index.css */
:root {
  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  background: #f5f7fb;
  color: #1f2937;
}

body { margin: 0; background: #f5f7fb; }
#root { min-height: 100vh; }
.app-shell { min-height: 100vh; display: flex; flex-direction: column; }
.app-main { flex: 1; }
.app-footer { padding: 18px; text-align: center; background: #212529; color: #f8f9fa; }
.card { border: 0; }
.card-title { color: #17365d; }
@media (max-width: 576px) {
  .app-footer { font-size: 0.85rem; }
}

```

Bước 26. Tạo IntelliJ npm Run Configuration

Mục	Hướng dẫn
ACTION	Run → Edit Configurations → Add npm (nếu edition/plugin hỗ trợ) → chọn package.json → command `run` → script `dev`.
WHY	Cho phép start app bằng IDE thay vì gõ lệnh mỗi lần.
EXPECTED RESULT	Run button khởi động Vite dev server.
HOW TO VERIFY	Stop → Run lại và mở URL.
IF FAILED	Nếu menu npm configuration không có, dùng Terminal `npm run dev`; đó vẫn là workflow hợp lệ.

Bước 27. Chạy Manual Test Matrix

Mục	Hướng dẫn
ACTION	Thực hiện test theo bảng, không chỉ “nhìn thấy app chạy”.
WHY	Verification evidence là một phần của năng lực engineering.
EXPECTED RESULT	Core tests PASS.
HOW TO VERIFY	Ghi Actual + PASS/FAIL.
IF FAILED	Fix first failing core test before optional polish.

ID	Action/Input	Expected	Actual	Result
T01	/	Home renders		
T02	Home → Explore Events	URL becomes /events		
T03	Navbar Events	EventsPage renders		
T04	/events	8 records initially		
T05	Search React	Matching event(s) only		
T06	Category Competition	Correct subset		
T07	Clear filters	8 records again		
T08	Event 1 detail	/events/1 + correct title		
T09	Event 3 detail	/events/3 + different data		
T10	/events/999	Resource not found		

ID	Action/Input	Expected	Actual	Result
T11	/abc123	404 client route		
T12	Back to Events Link	/events		
T13	Go Back	history works		
T14	Browser Back	previous view		
T15	Browser Forward	next view		
T16	Direct /events/4	detail loads in dev server		
T17	Mobile width	Navbar/grid readable		
T18	Console	No blocking errors		
T19	Terminal	No compile errors		
T20	Git status	Only intended files changed		

Bước 28. Debug theo Decision Tree

Mục	Hướng dẫn
ACTION	Khi lỗi, thu evidence: current URL, expected view, Routes config, Console, first terminal error, relevant component.
WHY	Routing bugs dễ bị che nếu sửa nhiều thứ cùng lúc.
EXPECTED RESULT	Xác định FIRST broken stage.
HOW TO VERIFY	Đi từ compile → Router context → URL → route match → params → data lookup → view.
IF FAILED	Không nhờ AI rewrite toàn project trước khi có evidence.

```

Navigation fails
↓
Does app compile?
↓
BrowserRouter present?
↓
Does URL change?
↓
Does a Route pattern match?
↓
Correct component rendered?
↓
If dynamic: useParams key/value correct?
↓
Does find() match ID type/value?
↓
Only then inspect UI details

```

Bước 29. Git + production build/preview

Mục	Hướng dẫn
ACTION	Sau khi verification matrix core PASS, commit; sau đó chạy build và preview.
WHY	Tạo working checkpoint và kiểm tra production bundle.
EXPECTED RESULT	Git log có commit; `npm run build` hoàn tất; preview chạy.
HOW TO VERIFY	`git status`, `git log -1`, build output, preview URL.
IF FAILED	Fix build error trước khi coi project hoàn thành.

```
git status
git add .
git commit -m "Add routed Campus Event Navigator SPA"
git log -1
```

```
npm run build
npm run preview
```

8. SOURCE CODE HOÀN CHỈNH — ĐỐI CHIẾU CUỐI

Các file code ở phần hướng dẫn chi tiết đã tạo thành một project hoàn chỉnh. Bảng sau dùng để rà soát file nào phải tồn tại và trách nhiệm của nó; không copy thêm cơ chế ngoài phạm vi Slot 07.

File	Trách nhiệm	API chính
src/main.jsx	Entry + Router context	BrowserRouter
src/App.jsx	Route composition	Routes, Route
src/data/events.js	Static local records	Array
components/AppNavbar.jsx	Internal navigation	Link via React-Bootstrap
components/EventCard.jsx	Reusable event summary	Props, Link
components/AppFooter.jsx	Shared layout	Component
pages/Home.jsx	Home + CTA	useNavigate
pages/EventsPage.jsx	Collection/filter	useState, filter, map, key
pages/EventDetail.jsx	Dynamic resource detail	useParams, useNavigate, find
pages/About.jsx	Static view	Route view
pages/NotFound.jsx	Client unknown route	Link
src/index.css	Basic layout/responsive	CSS

Completeness check

Nếu toàn bộ 12 file/module trên tồn tại đúng import path, dependencies đã cài và routes khai báo đúng, project phải có đủ chức năng core. Nếu không chạy, debug theo evidence; đừng tạo thêm library.

9. KIỂM TRA KIẾN THỨC ROUTING

9.1 Route Trace Worksheet

URL: /events/3
 Matching route pattern: _____
 Route parameter name: _____
 useParams value: _____
 Data lookup expression: _____
 Matched record: _____
 Rendered component: _____

9.2 Navigation Trace Worksheet

User action: _____
 Navigation API: Link / useNavigate / browser
 Target/new URL: _____
 Matching Route: _____
 Rendered View: _____

9.3 Client vs Server Trace

Case	Handled primarily by	Reason
React URL `/events/3` in this project	Client	React Router selects EventDetail
`events.find(...)`	Client	JavaScript reads local static data
Future `GET /events/3`	Server endpoint + client HTTP	Would retrieve data over HTTP
SQL query	Server/data layer	Not part of current frontend

10. COMMON ERRORS & CÁCH SỬA

Symptom	Likely cause	Evidence	Fix
Cannot find module react-router-dom	Package chưa cài / sai directory	package.json + terminal	npm install ở project root
useNavigate/useParams outside Router	BrowserRouter thiếu hoặc đặt sai	Console stack trace	Wrap App at entry point
URL changes but wrong/blank view	Path typo / import error	Address bar + Routes + Console	Compare URL with exact Route path
id is undefined	`:id` vs destructuring name mismatch	Route + `useParams()`	Use same parameter key
Event not found for valid ID	String/number mismatch or wrong field	log id + data id	Align types deliberately
All cards open same detail	Link constructed with wrong id	Inspect card URL	Use `event.id` per record
Navbar reloads unexpectedly	Using unintended normal href for internal route	Network/behavior	Use Link/`as={Link}`
/events/999 crashes	Reads event fields before null check	Console TypeError	Guard `if (!event)` first
Unknown URL shows nothing	Missing wildcard route	Routes config	Add `path="*"`
Refresh production path 404	Host lacks SPA fallback	works via navigation, fails refresh	Deployment/server config issue; not fixed by random React code

11. BUG INJECTION — HỌC BẰNG CÁCH LÀM HỎNG

- Đổi Link `/events` thành `/event` nhưng giữ Route cũ. Dự đoán kết quả, chạy, ghi evidence, sửa.
- Đổi route parameter thành `:eventId` nhưng vẫn `const { id } = useParams()`. Xác định first mismatch.
- Tạm đổi một data id từ string `"1"` thành number `1` và quan sát strict comparison với URL string.
- Xóa BrowserRouter, chạy và đọc chính xác console error; khôi phục.

5. Xóa wildcard route, gỡ `/abc`; mô tả difference giữa blank/no match và explicit NotFound UI.

Quy tắc

Mỗi bug injection phải có: Prediction → Evidence → Root Cause → Small Fix → Re-test. Không được chỉ “undo vì biết chỗ vừa sửa”.

12. HUMAN VERIFICATION GATE

Không dùng AI. Sinh viên phải trả lời/giải thích trực tiếp; UI chạy được nhưng không giải thích được thì chưa chứng minh năng lực Slot 07.

1. Routing là gì?
2. SPA có thể có nhiều view không?
3. BrowserRouter làm gì?
4. Routes và Route khác nhau thế nào?
5. ``path`` và ``element`` có ý nghĩa gì?
6. Link khác normal anchor thế nào?
7. Tại sao ``/events/:id`` là dynamic route?
8. ``useParams()`` trả gì trong ``/events/3``?
9. Route parameter thường là string hay number?
10. ``find()`` thuộc React Router hay JavaScript?
11. Tại sao một EventDetail xử lý được nhiều event IDs?
12. ``/events/999`` khác ``/invalid-page`` thế nào?
13. Wildcard ``*`` dùng làm gì?
14. `useNavigate` dùng khi nào?
15. Tại sao Link thường đơn giản hơn `useNavigate` cho menu?
16. Browser Back hoạt động dựa trên khái niệm gì?
17. React Router có tự lấy data từ Spring Boot không?
18. React route ``/events/1`` khác future ``GET /events/1`` thế nào?
19. Tại sao URL có thể xem như một phần application state?
20. Nếu direct refresh production route 404, em sẽ kiểm tra lớp nào ngoài React code?

13. EVIDENCE PACKAGE & RESULT ANALYSIS

Evidence	Nội dung phải nộp
E1 Route Design	Route table + static/dynamic classification
E2 Home	Screenshot <code>`/`</code>
E3 Collection	Screenshot <code>`/events`</code>
E4 Dynamic	Screenshot <code>`/events/2`</code>
E5 Invalid resource	Screenshot <code>`/events/999`</code>
E6 Invalid route	Screenshot <code>`/abc123`</code>
E7 Console	No blocking errors
E8 Git	Commit hash + message
E9 Explanation	URL → route → param → data lookup → rendered view

13.1 Result Analysis Form

PROJECT URL (DEV): _____

ROUTER
Entry point: _____
Router type: _____

ROUTES
Home: _____
Collection: _____
Detail pattern: _____
About: _____
NotFound: _____

DYNAMIC ROUTE
Parameter name: _____
Example value: _____
Data lookup: _____

NAVIGATION
Link test: PASS / FAIL
useNavigate test: PASS / FAIL
Back/Forward: PASS / FAIL
Direct URL: PASS / FAIL

GIT
Commit hash: _____

14. TỰ ĐÁNH GIÁ CHẤT LƯỢNG

Tiêu chí	0	1	2	3
Functional routing	Không chạy	Một phần route	Static routes chạy	Static + dynamic + not-found chạy
Component structure	Monolithic	Tách ít	Hợp lý	Responsibilities rõ + reuse
Navigation	Không có	href thủ công	Link	Link + justified useNavigate + history
Dynamic parameter	Không có	URL có nhưng không đọc	useParams đọc	lookup + invalid ID handling
Verification	Không evidence	Screenshot	Matrix một phần	Matrix + console + route trace
Explanation	Không giải thích	Nói API names	Giải thích flow	Phân biệt client route/server endpoint rõ

15. AI-AUGMENTED LEARNING — DÙNG AI SAU EVIDENCE

15.1 AI-0 trước code

Nhiệm vụ không AI

Với app có Home, Events, Event Detail, About: hãy tự thiết kế URL table, xác định static/dynamic route và parameter trước khi hỏi AI.

15.2 Prompt review route architecture

Act as a React Router tutor.

Requirements: [PASTE]

My route table: [PASTE]

My component tree: [PASTE]

Do NOT generate code. Review only:

1. route naming;
2. static vs dynamic route;
3. parameter justification;
4. missing NotFound;
5. duplicated view responsibility.

Keep scope to BrowserRouter, Routes, Route, Link, useParams, simple useNavigate.

15.3 Prompt debug useParams

Dynamic route: [PASTE]

Current URL: [PASTE]

useParams result: [PASTE]

Data IDs: [PASTE]

find() code: [PASTE]

Do not rewrite the component.

Check parameter name → value → type → data ID → comparison.

Identify the FIRST mismatch only.

15.4 Hallucination Check

Sai

“useParams() đọc record Event từ database.”

Sửa đúng

useParams() đọc route parameters từ URL. Trong project này, record được tìm trong mảng static `events`. Database/API sẽ học sau.

16. PROJECT TRANSFER — ÁP DỤNG VÀO ĐỀ TÀI NHÓM

Sau khi hoàn thành Campus Event Navigator, mỗi nhóm không copy domain Event mà chuyển pattern routing sang domain dự án thật.

Domain	Collection Route	Detail Route	Parameter
News	/news	/news/:id	id
Hotel	/rooms	/rooms/:id	id
Book	/books	/books/:id	id
Course	/courses	/courses/:id	id
Product	/products	/products/:id	id

Mini-deliverable nhóm

Route table + BrowserRouter + collection view + dynamic detail + useParams + NotFound + screenshots + Git commit. Chưa backend.

17. OPTIONAL CHALLENGES

1. Nested-route awareness: thiết kế `/account`, `/account/profile`, `/account/settings`; vẽ tree trước khi code.
2. Active navigation: nghiên cứu NavLink sau khi core project PASS; giải thích khác Link ở điểm nào.
3. Design a deployment note: BrowserRouter production refresh cần server fallback về index.html trong nhiều hosting setups. Chỉ mô tả, không đổi router nếu chưa có yêu cầu deployment.
4. Thay local `events` bằng một giả lập async data source chỉ ở mức thiết kế; không học Axios/REST trong challenge này.

Overengineering Guard

Không thêm Redux, Zustand, data router loaders/actions, auth guards, SSR, Next.js, Remix, Axios hoặc Spring Boot vào bài Slot 07.

18. QUICK COMMAND SHEET

```
# Environment
node -v
npm -v
git --version

# Dependencies
npm install
npm install react-bootstrap bootstrap react-router-dom

# Development
npm run dev

# Git checkpoint
git status
git add .
git commit -m "Add routed Campus Event Navigator SPA"
git log -1

# Production check
npm run build
npm run preview
```

19. FINAL COMPLETION CHECKLIST

- ☐ Project opens correctly in IntelliJ.
- ☐ Node/npm/Git verified.
- ☐ React-Bootstrap and react-router-dom installed.
- ☐ BrowserRouter wraps App.
- ☐ Routes contains five route patterns.
- ☐ Home route works.
- ☐ Events route works.
- ☐ About route works.
- ☐ EventCard reused with Props.
- ☐ map() uses stable `event.id` key.
- ☐ Search/category state works.
- ☐ Dynamic `/events/:id` works.
- ☐ useParams reads correct id.

- ☐ find() returns correct event.
- ☐ Invalid ID handled without crash.
- ☐ Wildcard NotFound works.
- ☐ Link navigation works.
- ☐ useNavigate example works.
- ☐ Browser Back/Forward checked.
- ☐ Direct dev URL checked.
- ☐ No blocking Console errors.
- ☐ No blocking Terminal errors.
- ☐ Manual core tests pass.
- ☐ Git checkpoint exists.
- ☐ Build completes.
- ☐ Student can distinguish client route from REST endpoint.

20. KẾT LUẬN & CẦU NỐI SANG BÀI TIẾP THEO

Sau Slot 07

Sinh viên phải có mental model: URL → React Router → matching Route → View; dynamic URL → useParams → local lookup → Detail. Router quyết định WHAT VIEW TO DISPLAY.

Bài tiếp theo

Dữ liệu hiện vẫn ở `events.js`. Bước tiếp theo là Client–Server Communication: React → HTTP Request → Server → HTTP Response/JSON → React. Khi đó cần phân biệt rõ client route `/events/3` với API request `GET /events/3`.

Khi chuyển sang HTTP/REST, không thay thế mental model routing; hai cơ chế phối hợp: Router chọn view, còn HTTP client lấy dữ liệu cần thiết cho view đó.